Елисеева Е.Р. 090303-ПИа-о24

Лабораторная работа №3

Объектно-ориентированное программирование в Python

Вариант № 5. Автопарк

1. Цель работы

Получить практические навыки разработки программ с использованием объектно-ориентированного подхода. В процессе выполнения лабораторной работы освоить понятия класса и объекта, атрибутов и методов, реализовать конструктор __init__, изучить инкапсуляцию (свойства property и setter), наследование, переопределение методов, полиморфизм и композицию объектов на языке программирования Python.

2. Постановка индивидуального задания

Разработать консольное приложение, моделирующее объекты заданной предметной области (Автопарк). Данные должны быть представлены объектами классов, а не словарями. Программа должна демонстрировать:

1) Создание базового класса Vehicle и не менее двух производных классов Car (Легковой автомобиль) и Truck (Грузовой автомобиль);

2) Хранение состояния объектов, использование и переопределение методов (метод расчета транспортного показателя get_performance);

3) Реализацию инкапсуляции: минимум один атрибут (цена) должен быть защищен от неконтролируемого изменения с использованием _price и свойства property/setter;

4) Демонстрацию полиморфизма посредством создания коллекции объектов разных производных классов и вызова общего метода в одном цикле;

5) Демонстрацию композиции: класс Fleet (Автопарк) должен содержать список объектов Vehicle (отношение HAS-A);

6) Реализацию интерактивного консольного меню для взаимодействия с пользователем.

3. Краткие теоретические сведения

Объектно-ориентированное программирование (ООП) — подход к разработке программ, при котором программа представляется как совокупность взаимодействующих объектов. Объект объединяет состояние (данные/атрибуты) и поведение (методы). Класс является описанием структуры и поведения объектов.

Инкапсуляция предполагает объединение данных и методов работы с ними внутри объекта и ограничение неконтролируемого доступа к внутреннему состоянию. В Python для контроля доступа применяются свойства (@property и @property.setter).

Наследование позволяет создать новый класс на основе существующего, перенимая его атрибуты и методы. Для вызова конструктора родительского класса используется super(). Переопределение методов позволяет изменить реализацию метода родительского класса в производном классе.

Полиморфизм позволяет работать с объектами разных классов через общий интерфейс (одинаковые имена методов), при этом результат работы метода зависит от фактического типа объекта. Композиция (отношение HAS-A) означает, что один объект содержит внутри себя другой объект в качестве составной части.

4. Исходный код программы

```
def get_num_input(prompt, is_float=False):
    while True:
        try:
            return float(input(prompt)) if is_float else
int(input(prompt))
        except ValueError:
            print("ошибка. введите числовое значение")


class vehicle:
    def __init__(self, brand, model, price):
        self.brand, self.model = brand, model
        self.price = price
    @property
```

```
    def price(self):
        return self._price
    @price.setter
    def price(self, value):
        if value < 0:
            raise ValueError("цена не может быть отрицательной!")
        self._price = value
    def get_performance(self):
        return 0


class car(vehicle):
    def __init__(self, brand, model, price, passengers):
        super().__init__(brand, model, price)
        self.passengers = passengers
    def get_performance(self):
        return round(self.price / self.passengers, 2)
    def __str__(self):
        return f"легковой: {self.brand} {self.model} ; цена: {self.price}
руб. ; мест: {self.passengers}"


class truck(vehicle):
    def __init__(self, brand, model, price, capacity):
        super().__init__(brand, model, price)
        self.capacity = capacity
    def get_performance(self):
        return round(self.price / self.capacity, 2)
    def __str__(self):
        return f"грузовой: {self.brand} {self.model} ; цена: {self.price}
руб. ; грузоподъемность: {self.capacity} т."


class fleet:
    def __init__(self, name):
        self.name, self.vehicles = name, []


def main():
    my_fleet = fleet("автопарк")
    my_fleet.vehicles = [
        car("toyota", "camry", 2000000, 5),
        truck("KAMAZ", "65115", 5000000, 15),
        car("lada", "vesta", 1200000, 5),
    ]
    while True:
        print(f"\n===== МЕНЮ АВТОПАРКА '{my_fleet.name}' =====")
        print("1. добавить | 2. показать все | 3. действие | 4. изменить
цену | 5. удалить | 6. статистика | 0. выход")
        choice = input("выберите действие: ").strip()
        if choice == "1":
            t_type = input("тип (1 - легковой, 2 - грузовой): ").strip()
            if t_type not in ("1", "2"):
                print("неверный тип!")
                continue
            brand, model = input("марка: ").strip(), input("модель:
").strip()
            price = get_num_input("цена (руб.): ", is_float=True)
            if t_type == "1":
```

```
                p = get_num_input("количество пассажиров: ")
                my_fleet.vehicles.append(car(brand, model, price, p))
            else:
                c = get_num_input("грузоподъемность (тонн): ",
is_float=True)
                my_fleet.vehicles.append(truck(brand, model, price, c))
            print("транспорт добавлен!")
        elif choice == "2":
            if not my_fleet.vehicles:
                print("автопарк пуст.")
            for idx, v in enumerate(my_fleet.vehicles, 1):
                print(f"{idx}. {v}")
        elif choice == "3":
            print("\n--- транспортные показатели (стоимость единицы
емкости): ---")
            for v in my_fleet.vehicles:
                print(
                    f"- {v.brand} {v.model}  показатель:
{v.get_performance()} руб."
                )
        elif choice == "4":
            try:
                idx = get_num_input("введите номер транспорта: ") - 1
                if 0 <= idx < len(my_fleet.vehicles):
                    my_fleet.vehicles[idx].price = get_num_input(
                        "новая цена: ", is_float=True
                    )
                    print("цена успешно изменена!")
                else:
                    print("неверный номер!")
            except ValueError as e:
                print(f"ошибка изменения: {e}")
        elif choice == "5":
            idx = get_num_input("введите номер для удаления: ") - 1
            if 0 <= idx < len(my_fleet.vehicles):
                print(
                    f"удален: {my_fleet.vehicles.pop(idx).brand}"
                )
            else:
                print("неверный номер!")
        elif choice == "6":
            if not my_fleet.vehicles:
                continue
            prices = [v.price for v in my_fleet.vehicles]
            print(
                f"\nвсего техники: {len(my_fleet.vehicles)} ед.\nобщая
стоимость: {sum(prices)} руб.\nсредняя цена: {round(sum(prices)/
len(prices), 2)} руб."
            )
        elif choice == "0":
            print("работа программы завершена")
            break


if __name__ == "__main__":
    main()
```

5. Результаты тестирования

```
elizavetaeliseeva@EER-3 090303-ПИа-о24 Елисеева Е.Р. %  source "/Users/EER/Desktop/090303-ПИа-о24 Елисеева Е.Р./venv/bin/activate"
(venv) elizavetaeliseeva@EER-3 090303-ПИа-о24 Елисеева Е.Р. % "/Users/EER/Desktop/090303-ПИа-о24 Елисеева Е.Р./venv/bin/python" "/Users/EER/Desktop/090303-ПИа-о24 Елисеева
.P./third.py"

===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 1
тип (1 – легковой, 2 – грузовой): 1
марка: ford
модель: focus
цена (руб.): 1200000
количество пассажиров: 4
транспорт добавлен!

===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 1
тип (1 – легковой, 2 – грузовой): 2
марка: mercedes
модель: benz
цена (руб.): 6000000
грузоподъемность (тонн): 20
транспорт добавлен!
```

В соответствии с требованиями раздела 12 методических указаний, было проведено комплексное тестирование разработанной объектной модели по 7 контрольным сценариям.

Тест 1. Создание объектов каждого типа

Описание: Выбор пункта 1 меню, поочередное создание экземпляра класса car и класса truck. Объекты успешно инициализируются, это показано на рисунке 3.1

```
===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 2
1. легковой: toyota camry ; цена: 2000000 руб. ; мест: 5
2. грузовой: KAMAZ 65115 ; цена: 5000000 руб. ; грузоподъемность: 15 т.
3. легковой: lada vesta ; цена: 1200000 руб. ; мест: 5
4. легковой: ford focus ; цена: 1200000.0 руб. ; мест: 4
5. грузовой: mercedes benz ; цена: 6000000.0 руб. ; грузоподъемность: 20.0 т.
```

Рисунок 3.1 - создание объектов

Тест 2. Корректное выполнение общих методов

```
===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 3

--- транспортные показатели (стоимость единицы емкости): ---
- toyota camry  показатель: 400000.0 руб.
- KAMAZ 65115  показатель: 333333.33 руб.
- lada vesta  показатель: 240000.0 руб.
- ford focus  показатель: 300000.0 руб.
- mercedes benz  показатель: 300000.0 руб.
```

Описание: Выбор пункта 2 меню для демонстрации созданных объектов. Метод __str__ автоматически форматирует вывод данных. Второй тест приведен на рисунке 3.2

Рисунок 3.2 - показ всех объектов

Тест 3. Работа переопределенных методов

Описание: Выбор пункта 3 меню приведен на рисунке 3.3. Метод get_performance() возвращает разные логические результаты для легковых автомобилей (цена/места) и для грузовиков (цена/тонны).

Рисунок 3.3 - транспортные показатели

Тест 4. Изменение состояния объекта

Описание: Выбор пункта 4 меню (см. Рисунок 3.4). Пользователь указывает номер существующей машины и успешно задает ей новое корректное числовое значение цены.

```
===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 4
введите номер транспорта: 4
новая цена: 1300000
цена успешно изменена!
```

Рисунок 3.4 - изменение цены

Тест 5. Обработку некорректного значения свойства

Описание: Выбор пункта 4 меню и попытка ввести отрицательное значение цены (например, -500) приведены на рисунке 3.5. Срабатывает условие в блоке @price.setter, выбрасывается исключение ValueError, программа перехватывает его и выдает текстовое предупреждение, не завершая свою работу.

```
===== МЕНЮ АВТОПАРКА 'автопарк' =====
1. добавить | 2. показать все | 3. действие | 4. изменить цену | 5. удалить | 6. статистика | 0. выход
выберите действие: 4
введите номер транспорта: 4
новая цена: -500
ошибка изменения: цена не может быть отрицательной!
```

Рисунок 3.5 - некорректное значение

Тест 6. Работу полиморфной коллекции

Описание: Продемонстрировано в Тесте 3. Объекты классов car и truck находятся в едином списке my_fleet.vehicles и обрабатываются в одном цикле for через общий вызов get_performance().

6. Вывод

В процессе выполнения лабораторной работы №3 были успешно получены практические навыки применения объектно-ориентированного программирования (ООП) на языке Python. Была спроектирована и реализована устойчивая объектная модель предметной области 'Автопарк' с использованием базового класса, механизмов наследования и полиморфизма. Реализована надежная инкапсуляция критических данных с помощью свойств @property и @price.setter, предотвращающая ввод некорректного состояния объектов. Применение композиции позволило эффективно структурировать данные и связать сущности в единую логическую систему, управляемую через интерактивное консольное меню в циклической структуре while.